Different families of projections have different scale defaults. The `d3.geo.albersUsa` projection defaults to 1070, while `d3.geo.mercator` defaults to 150. As with most D3 functions like this, you can see the default by calling the function without passing it a value:

```
d3.geo.mercator().scale()
d3.geo.albersUsa().scale()
```

150

1070

By adjusting the `translate` and `scale` as in listing 7.5, we can adjust the projection to show different parts of the geodata we're working with—in our case, the world. The result in figure 7.4 shows that we now see the entire world rendered.

Listing 7.5 Simple map with scale and translate settings

```
function createMap(countries) {
  var width = 500;
  var height = 500;
  var aProjection = d3.geo.mercator()
  .scale(80)
  .translate([width / 2, height / 2]);
  var geoPath = d3.geo.path().projection(aProjection);

  d3.select("svg").selectAll("path").data(countries.features)
   .enter()
   .append("path")
   .attr("d", geoPath)
   .attr("class", "countries");

};
```

By defining the size of our SVG as variables, we can refer to them throughout our visualization code.

Moves the center of the projection to the center of our canvas

Scale values are different for different families of projections; 80 works well in this case.



Figure 7.4 The Mercator-projected world from our data now fitting our SVG area. Notice the enormous distortion in size of regions near the poles, such as Greenland and Antarctica.